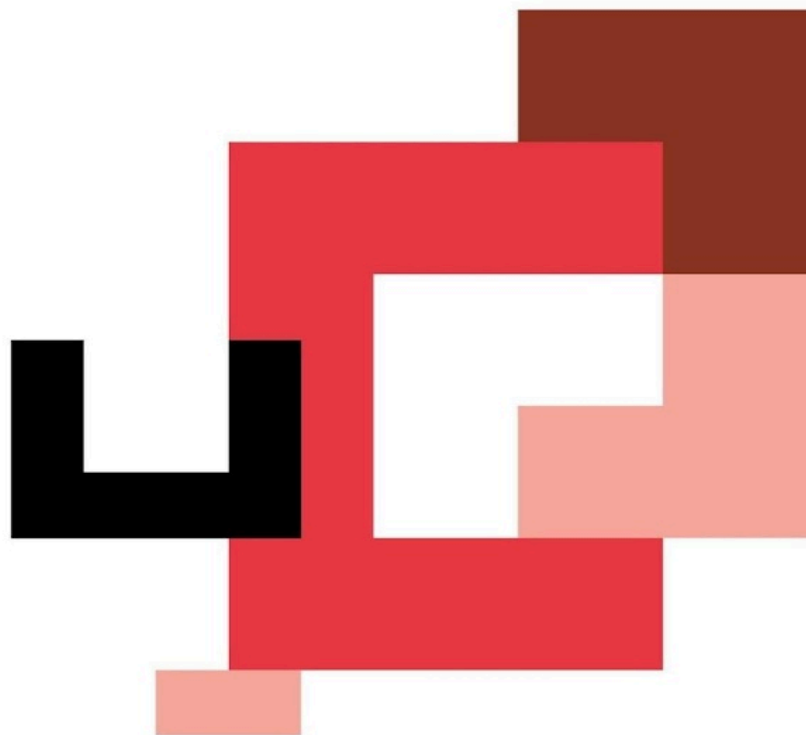




FULL STACK DEVELOPER + IA

Módulo 1 | Feature 4

Generando dinamismo



Objetivo de la entrega: El objetivo central de este Sprint es transformar el proyecto del Sprint 3 añadiendo **funcionalidad JavaScript** para hacerlo dinámico e interactivo. Esto incluye la implementación de módulos reutilizables, un sistema de tema (modo oscuro) persistente, la carga dinámica de habilidades (*skills*) y un menú de navegación adaptable a dispositivos móviles.

Aquí tienes el ejemplo de cómo debería quedar aproximadamente. El tuyo deberá ser con tu información e imágenes. Navega por la página, reduce su tamaño para ver el responsive y explora cada parte para entender mejor que tienes que hacer.

<https://carlosdiazgirol.github.io/proyecto1/sprint4/index.html>

Partiendo de la estructura existente del Sprint 3, se debe añadir la siguiente estructura modular dentro de una carpeta principal para el código JavaScript:

```
.
├── js/
│   ├── templates/      # Componentes HTML reutilizables (Header,
Footer)
│   ├── utils/          # Funcionalidades (Modo Oscuro, Menú, Render
de Skills)
│   ├── data/           # Datos estáticos (Skills)
│   └── main.js         # Punto de entrada de la aplicación
```

Archivos JavaScript Requeridos

Archivo	Rol y Requisitos
js/templates/template.js	Módulos Reutilizables: Exportar constantes <i>header</i> y <i>footer</i> como <i>template strings</i> . El <i>header</i> debe incluir el Logo, el menú de navegación (<i>nav-links</i>), el botón de modo oscuro (con iconos sol/luna), y el botón hamburguesa .
js/utils/darkMode.js	Modo Oscuro: Exportar la función <i>darkMode</i> (por defecto). Debe implementar la lógica de <i>toggle</i> para cambiar entre modo claro/oscuro, añadir/quitar la clase <i>'dark-mode'</i> al elemento que contenga toda la página. Siguiente Sprint: Guardar la preferencia en <i>localStorage</i> y cargar esta preferencia al iniciar la aplicación.

js/utils/menu.js	Menú Hamburguesa: Exportar la función <code>menu</code> (por defecto). Implementar el <code>toggle</code> para la hamburguesa, añadir/quitar la clase <code>'active'</code> a la hamburguesa y al menú (<code>nav-links</code>), y cerrar automáticamente el menú al hacer clic en un enlace.
js/data/skills.js	Datos de Skills: Exportar un <i>array de objetos</i> con las habilidades. Mínimo 4 skills con el estado <code>"dominada"</code> (ej: HTML, CSS, JS, GIT). Cada objeto debe incluir: <code>name</code> , <code>icon</code> (URL), <code>state</code> , <code>level</code> , <code>description</code> y <code>tags</code> (array de strings).
js/utils/skillsList.js	Renderizado de Skills: Importar los datos de <code>skills.js</code> . Filtrar solo las <i>skills</i> con <code>state: "dominada"</code> . Generar el HTML dinámicamente utilizando métodos de array (<code>map()</code>). Renderizar el resultado en la sección de <i>skills</i> del <code>index.html</code> (mostrar icono, nombre, nivel, descripción, tags).
js/main.js	Inicialización: Importar <code>header</code> y <code>footer</code> de <code>template.js</code> y todas las funcionalidades de <code>utils/</code> . Cargar los templates en los elementos correspondientes del DOM e inicializar todas las funcionalidades (<code>darkMode()</code> , <code>menu()</code> , <code>skillsList()</code>).

Modificaciones en HTML

- Eliminar el código HTML del `header` y `footer` en todas las páginas, ya que será cargado dinámicamente por `main.js`.
- Añadir etiquetas con `id` o clases específicas en `index.html` para que `main.js` pueda inyectar el `header`, el `footer` y la lista de *skills*.
- Asegurar que los *scripts* de JavaScript se carguen correctamente, usando la propiedad `type="module"` en la etiqueta de `main.js`.

Pistas y Consideraciones Clave

1. Sistema de Módulos (ES6)

- **Pista:** Utiliza `export` e `import` en cada archivo JS. Recuerda que para que esto funcione, debes añadir el atributo `type="module"` a la etiqueta `<script>` que enlaza a `main.js` en tu HTML.

```
<script type="module" src="./js/main.js"></script>
```

2. Manipulación del DOM

- **Consideración:** Usa `document.querySelector()` o `document.getElementById()` para obtener los elementos donde inyectarás los templates (`header` y `footer`) y la lista de `skills`. Para inyectar el HTML de las plantillas, `element.innerHTML = templateString` es la opción más sencilla.

3. Modo Oscuro con Persistencia

- **Pista:** El estado del modo oscuro (claro/oscuro) debe guardarse en el navegador. Usa `localStorage.setItem('theme', 'dark')` y `localStorage.getItem('theme')` para almacenar y recuperar la preferencia del usuario. **Esto aún no es necesario.** Si te entra la intriga puedes investigar como hacerlo. Sino será una tarea para la siguiente pieza.
- **Consideración:** La función `darkMode` en `main.js` debe llamarse **inmediatamente** al cargar la página para aplicar el tema guardado antes de que el usuario vea el contenido.

4. Iconos de Skills

- **Pista:** Para obtener las URLs de los iconos de `skills`, puedes seguir el patrón del ejemplo: `https://cdn.simpleicons.org/{nombre-del-icono}/{color-hexadecimal}`. Por ejemplo, para el icono de **Git** en color negro: `https://cdn.simpleicons.org/git/000000`.

En <https://simpleicons.org> podrás buscar los iconos.

Piensa si la IA te puede ayudar a crear un array de objetos diciéndole lo que quieres. Este es el ejemplo de objeto:

```
{
  name: "HTML5",
  icon: "https://cdn.simpleicons.org/html5/E34F26",
  state: "dominada",
  level: "avanzado",
  description: "Estructura semántica y accesible para aplicaciones web.",
  tags: ["Semantic HTML", "Accessibility", "SEO"]
},
```

5. Renderizado Dinámico

- **Consideración:** En `skillsList.js`, el método `map()` es ideal para transformar el array de datos (`skills`) en un array de *template strings* (un *template string* por cada *skill*). Luego, usa el método `join("")` para unir ese array de strings en una única cadena de HTML antes de inyectarla en el DOM.

Uso de Inteligencia Artificial para Revisión de Código

Una vez finalizado el desarrollo de JavaScript, se recomienda utilizar **GitHub Copilot** para validar la implementación, la estructura y las buenas prácticas del código.

Prompt Sugerido para la Revisión:

"Revisa mi código JavaScript y dime si he implementado correctamente:

1. Sistema de módulos ES6 con import/export.
2. Templates reutilizables para header y footer.
3. Modo oscuro cambio de iconos.
4. Renderizado dinámico de skills desde array de datos.
5. Toggle del menú hamburguesa con animaciones.
6. Event listeners eficientes.
7. Si hay redundancias o malas prácticas.

Importante:

- **NO** copies código de la IA sin entender su funcionamiento.
- Prueba cada funcionalidad (*Modo Oscuro, Menú Hamburguesa, Skills List*) por separado.
- Usa `console.log()` para depurar y verificar que los datos y los módulos se importen correctamente.